



Text Analysis and Natural Language

Processing

Project Report

‘ToxiGuard’

Author: Amine Habbou

Professors: Wilhelm Adalbert and

Meckel Matthias

26/4/ 2026

Outline

Abstract.....	3
1. Introduction.....	4
1.1 Statement of Problem and Motivation.....	4
1.2 Research goals.....	4
1.3 Contributions and scope of the research.....	5
2. Related Work.....	5
2.1 Rule-based and Lexical Approaches.....	5
2.2 Classical Machine Learning for Toxicity Detection.....	6
2.3 Deep Learning Approaches.....	6
2.4 Transformer-Based Models.....	6
2.5 Detoxification and Generative Approaches.....	7
2.6 Benchmark Datasets.....	7
3. Methodology.....	7
3.1 Data Collection and Preprocessing.....	7
3.1.1 Source Selection.....	7
3.1.2 Preprocessing Pipeline.....	9
3.2 Exploratory Data Analysis.....	10
3.2.1 Text Length Analysis.....	10
3.2.2 Punctuation and Capitalization Analysis.....	11
3.2.3 Sentiment Analysis.....	12
3.2.4 Readability Analysis.....	13
3.2.5 Lexical Analysis.....	13
3.3 Feature Engineering and Correlation-Based Selection.....	15
3.3.1 Engineered Features.....	15
3.3.2 Correlation Analysis.....	16
3.3.3 Random Forest Feature importance.....	20
3.4 Baseline Model – Logistic Regression (TF-IDF).....	20
3.5 Optimized Classical Models (SVM, KNN, Random Forest).....	20
3.5.1 SVM.....	21
3.5.2 KNN.....	22
3.5.3 Random Forest.....	22
3.6 ToxiGuard Architecture.....	22
3.6.1 Bert Fine-Tuning (Classification Head).....	22
3.6.2 GPT-2 Fine-Tuning (Explanation & Alternative Head).....	22
3.6.3 Combined ToxiGuard Pipeline.....	23
3.7 Code Availability.....	24
4. Experiments and Results.....	24
4.1 Experimental Setup.....	24
4.2 Classical Models – Quantitative Results.....	25
4.3 ToxiGuard – Quantitative Results.....	26
4.4 Model Comparison.....	27
4.5 Qualitative Analysis & Examples.....	28
4.5.1 Correct Classifications with GPT-2 Output.....	28
4.5.2 Failure Cases.....	29
5. Discussion.....	30
5.1 Interpretation of Results.....	30
5.2 Limitations.....	30
5.3 Ethical considerations.....	31
6. Conclusion & Future Work	31
7. References.....	32

Abstract

The emergence of toxicity in online spaces has become an increasing problem for digital platforms. As a result of the need for automated detection systems for online toxicity that are both accurate and interpretable, the ToxiGuard end-to-end natural language processing (NLP) pipeline was developed to detect toxic comments, explain the harmful aspects of these comments and provide constructive alternatives.

This system was developed using a total of 52,768 comments from 3 publicly available sources: the Google Jigsaw Civil Comments dataset, the Davidson Hate Speech and Offensive Language dataset and the ParaDetox corpus. This combined dataset provides a broad representation of both toxic language and non-toxic language.

The research was based upon a 4-stage modeling process. In the first stage, a baseline model using TF-IDF features and Logistic Regression produced an accuracy rate of 92.37% and a macro F1 score of 0.924 on a balanced test sample of 10,005. In the second stage, a set of handcrafted syntactic and semantic features, including sentiment score, readability, length of text, and punctuation, were created. The handcrafted features were then utilized to train classical machine learning algorithms (e.g. SVM, Random Forest, KNN) to develop the model. Among the models tested, the SVM emerged as having the highest accuracy at 94.29% and an area under the curve (AUC) of 0.978.

After creating the ToxiGuard system, it was determined that the third step involved using the BERT-base model to directly fine-tune the raw text. When applying this additional training to the ToxiGuard models, an impressive increase in accuracy (96.074%) and a macro F1-score (0.961) were achieved compared to all previously constructed feature-based approaches.

Once both of these systems were created, they were integrated into the ToxiGuard architecture. The fine-tuned BERT model will serve as the classification module, while the fine-tuned GPT-2 model will generate structured, natural language explanations and constructive rewrites for toxic comments identified by ToxiGuard.

From further analysis of the ToxiGuard models, it can be determined that sentiment (e.g., negative) is the strongest predictor of toxicity, with a Pearson correlation coefficient of 0.46. In contrast, the longer and more complex the syntax of a comment, the more likely it is that the comment is not toxic. The quantitative analysis of the importance of engineered features compared to the typical indicators (e.g., profanity, etc.) of a toxic comment also supports this conclusion.

Through qualitative evaluation of the GPT-2 model as the generative component of ToxiGuard, it has been demonstrated that the GPT-2 model provides coherent explanations of toxicity and can effectively rewrite or reformulate toxic comments into more constructive alternatives. Overall, ToxiGuard is an interpretable and robust system that can effectively classify toxic comments while also providing action-oriented and human-readable feedback.

1. Introduction

1.1 Statement of Problem and Motivation

The phenomenal increase of user generated content on social media networks, forum comments, and in general online community environments have enabled unprecedented amounts of text communication. The growth of these communication channels also allowed for more extensive participation and sharing of ideas, at the cost of widely available toxic language, such as hate speech, harassment, personal attacks, and so on. Considering the enormous scale of text content generated everyday (billions of messages, in social media alone), manually monitoring every comment is simply impossible, which is why automated systems for toxicity detection are necessary.

However, toxicity detection isn't always enough. Most toxicity detection models are built on binary or categorical classifiers and do not give an explanation for why a text is considered toxic. This leads to opacity in content moderation, where users may feel that the moderating decision is arbitrary and lose their motivation to change their language use. Studies show that without a clear explanation of their mistake, users are more likely to contest a moderation decision and less likely to adapt their language. As such, there is a large gap between the identification and the explanation of toxic content.

The purpose of the current project, called ToxiGuard, is to fill that gap. Instead of classification, we want to use explanation through natural language generation and rewrites. We want to maintain user intent, and merely remove any malicious elements. We see toxicity moderation not as purely a penalty, but an educational process where the user is given feedback about what they have done wrong. To achieve this goal, we require a combination of powerful classifiers and generative language abilities, creating a hybrid model.

1.2 Research Goals

In this project, we address four research goals, each one dependent on the previous step.

First, we build a comprehensive multi-source toxicity classification dataset consisting of diverse toxic and non-toxic text. The combined data comes from three sources: Jigsaw Civil Comments dataset, Davidson's hate speech dataset and ParaDetox. We achieve to minimize the impact of domain bias.

Second, we performed a full exploratory data analysis (EDA) and feature engineering. It's important to understand what attributes, like length, sentiment, readability, punctuation, or capitalization, contribute to making a text toxic. The correlations of these attributes are computed to find meaningful features for classical machine learning models.

Third, we comparatively analyze different models. We start by a TF-IDF based Logistic Regression baseline, then three classical machine learning models, which were trained on our engineered features: Support Vector Machine (SVM), Random Forest, K-Nearest Neighbours (K-NN). Further on we fine-tune a transformer model (BERT), and finally we construct our full hybrid model.

Fourth, and as our most novel contribution, we develop a generative component that gives explanations for and rewrites the toxic comments. Using a fine-tuned GPT-2 model, with some template generation logic, we aim to both explain why the comments are toxic and rewrite them. The explanation and rewrite generation is evaluated qualitatively, but the whole model performance relies on a good classification part.

1.3 Contributions and scope of the research

In this research we contributed the following:

First of all, we developed a reproducible data pipeline that concatenates, cleanses, de-duplicates and balances three distinct datasets, into one corpus of 50,022 samples, along with explicit description of the pre-processing steps involved.

Second, we executed a full EDA analysis and demonstrated what features differ between toxic and non-toxic content, based on their structure, lexis, sentiment, readability and other features. We also examined correlations between our engineered features.

Third, we analyzed the inter-relation between the features. We used three different correlation measures (Pearson, Spearman, and Kendall) to assess the relation of the 15 engineered features. Based on them, we also show a rank ordering of features by their importance on the best performing classical model.

Fourth, we proposed ToxiGuard, a hybrid model composed of two parts, a BERT-based toxicity classifier and a GPT-2 based generative explanation module and rewrite. Our classification module achieves 96.07% accuracy and macro F1 score of 0.96. This hybrid system was evaluated on a held-out test set of 10,554 samples.

This paper is organized as follows: Section 2 provides a literature review on toxicity detection, hate speech classification and generation. Section 3 describes the methodology used in the experiment including the data processing as well as the model implementation. Section 4 displays and explains the results from experiments on a test set. Section 5 discusses limitations and ethics related to the project, and finally, section 6 summarizes the outcomes and highlights future directions for research.

2. Related Work

2.1 Rule-based and Lexical Approaches

The first automated systems for detecting toxic content were based on hand-crafted lexicons and pattern-matching rules. These systems maintain lists of "bad words" and simply flag any text containing them, potentially adding weights depending on severity or context. They are simple and easily interpretable, but also notoriously simple to circumvent by substituting characters (e.g. "asshole" becoming "4\$\$h0le"), misspellings or sarcasm, and have very high false-positive rates on legitimate uses (e.g., discussion of racist language history). This led to a shift towards statistical learning approaches that use context instead of solely vocabulary membership, although rule-based methods are often used as features for deeper models. Indeed, word frequency visualisations for

our training corpus show a majority of the most discriminative words were profane slang, which points to the efficacy and limitations of using vocabulary for toxicity detection in isolation.

2.2 Classical Machine Learning for Toxicity Detection

TF-IDF (Term Frequency-Inverse Document Frequency) vectorisation schemes combined with linear classifiers offered a significant improvement over pure lexical matching. Logistic Regression trained on TF-IDF inputs proved to be efficient, scalable and interpretable (through weights assigned to learned features). Nobata et al. (2016) found they could achieve competitive performance using handcrafted features on Yahoo comments without using deep learning models: their approach combined n-gram inputs with metadata such as comment length and number of sentences. Their results demonstrated the predictive power of simple, surface-level textual features. In the early 2010s linear Support Vector Machines were extensively used for detecting hate and offensive language, including one of the data sources used in this project (Davidson et al., 2017), which used linear SVM on TF-IDF characters, POS tags and sentiment scores to achieve highly accurate toxicity classification over 3 classes (hate, offensive, or none). These approaches are informative for our feature engineering strategy (Section 3.3). Random Forest and gradient-boosting ensemble methods were also found useful for capturing non-linear dependencies and obtaining feature importance. A problem with all classical approaches is their dependency on training data characteristics, such that a model trained on twitter language may fail when classifying professional communication. This motivates the usage of mixed datasets for training in this project.

2.3 Deep Learning Approaches

The availability of large datasets and computing resources allowed for deeper learning models that can represent words and sentences with distributed embeddings (word2vec, GloVe). Convolutional Neural Networks (CNNs) learned representations of local n-gram contexts and performed well on classifying short texts, whereas Long Short-Term Memory networks (LSTMs) capture longer-range dependencies and improve modeling negation and sarcasm by considering sequences of words. Trained on datasets for the Jigsaw Toxic Comment Classification Challenge (Kaggle, 2018), bi-directional LSTMs coupled with attention mechanism models attained SOTA performance. These works showed that contextual representations-where words are contextualised based on surrounding text-are critical for modeling toxic language. However, training large deep learning models from scratch on specific datasets requires significant annotated data and is sensitive to hyperparameter settings.

2.4 Transformer-Based Models

The transformer architecture with a bidirectional pre-training scheme, first described by Devlin et al. (2019) as BERT (Bidirectional Encoder Representations from Transformers), redefined NLP research. After pre-training over large corpora on tasks like Masked Language Model and Next Sentence Prediction, the deep transformer model can be fine-tuned for task-specific applications with limited labeled data. Toxicity classification using fine-tuned BERT-base already surpasses classical and task-specific deep learning models on several benchmarks, with Detoxify (Hanu & Unitary, 2020) reporting strong language-agnostic multi-label toxicity classification performance.

The Perspective API deployed by Jigsaw and Google leverages a transformer-based approach for production toxicity detection. These works have highlighted that transformer models capture contextual representations at SOTA level, motivating their use for classification head of ToxiGuard. XLNet and RoBERTa offer potential benefits with RoBERTa outperforming BERT-base at classifying longer documents (likely due to removal of next sentence prediction task). We selected BERT-base-uncased here for ease of use and reproducibility while still capturing good representations of the text.

2.5 Detoxification and Generative Approaches

Detoxification – generating a less harmful rewrite of toxic content that maintains the original message-is an area that is not as explored as classification. The ParaDetox dataset (Dale et al., 2021), one of the 3 datasets in this project, was designed to facilitate supervised detoxification by including pairs of toxic and their non-toxic rewrites, which we used as a training source for both components of ToxiGuard. We utilized a classifier-guided decoding scheme with conditional text generation. Style transfer approaches are also explored using GPT-2-one of the strong generative models-fine-tuned for content rewriting (Yi et al., 2021; Logacheva et al., 2022). They noted that controlled prompts led to relevant and useful suggestions, so we also used templates to prompt GPT-2 for a structured output of both explanations and concrete rewrite suggestions. We chose not to use RLHF, as it relies on significant annotation and computing resources that are out of scope for this project. We aim for a pragmatic model consisting of a fine-tuned BERT classifier and a fine-tuned GPT-2 generator.

2.6 Benchmark Datasets

The three datasets used in this project-Civil Comments (Borkan et al., 2019), Hate Speech (Davidson et al., 2017) and ParaDetox (Dale et al., 2021)-complement each other: Civil Comments uses news comments, focuses on sub-group identity, and includes roughly balanced toxic/civil class distribution; Hate Speech comes from twitter and is highly unbalanced (84% toxic); ParaDetox provides pairs of toxic and civil text for supervised detoxification. Given the variance of language on the web, each dataset covers a distinct part of the spectrum of online toxicity. Our model aims to benefit from these diverse inputs to provide more robust, out-of-domain toxicity detection.

3. Methodology

This section describe the entire ToxiGuard pipeline in the order of each step being implemented. Figure numbers refer to the plots created as part of the Exploratory Data Analysis (EDA) and modeling processes.

3.1 Data Collection and Preprocessing

3.1.1 Source Selection

The decision for the three public datasets that make up the ToxiGuard training corpus is made based on three constraints: (a) that there is clear labels that either can be used directly or can be transformed into a binary class label, (b) the source must be accessible under a license that supports

research, (c) it's diverse in both the subject-domain and language style used so that not the bias from just one source is applied to the corpus.

From the Google Jigsaw Civil Comments dataset, 20,000 comments from online news sites are gathered, where each comment has a toxicity score in $[0, 1]$. For the binary classification problem, comments with a toxicity score higher than 0.5 were labeled as toxic. That would mean 5% positives in the dataset, a naturally low number because it's typically the duty of moderated discussion forums to maintain low proportions of toxic content.

From the Davidson Hate Speech and Offensive Language dataset, 24,783 tweets are collected. Each tweet has a label of Hate speech, Offensive language, or None. To make the labels fit the objective of a binary classifier, both Hate speech and Offensive language labels were combined to get one Toxic label class. That results in about 84% of positive examples, a high imbalance compared to the Jigsaw dataset.

From the ParaDetox corpus, about 8,200 sentences are retrieved where the sentences are made of pairs of an original toxic statement and its neutralized rewrite. This would directly support the generative component since it models a transition from toxic to clean language.

Dataset	Samples	Toxic (%)	Avg. Length (words)	Domain
Jigsaw Civil Comments	20,000	~5%	~57	News comments
Hate Speech (Davidson)	24,783	~84%	~14	Twitter / social media
ParaDetox	~8,200	50%	~10	Parallel detoxification
Total (after cleaning)	52,768	47.4%	—	Multi-domain

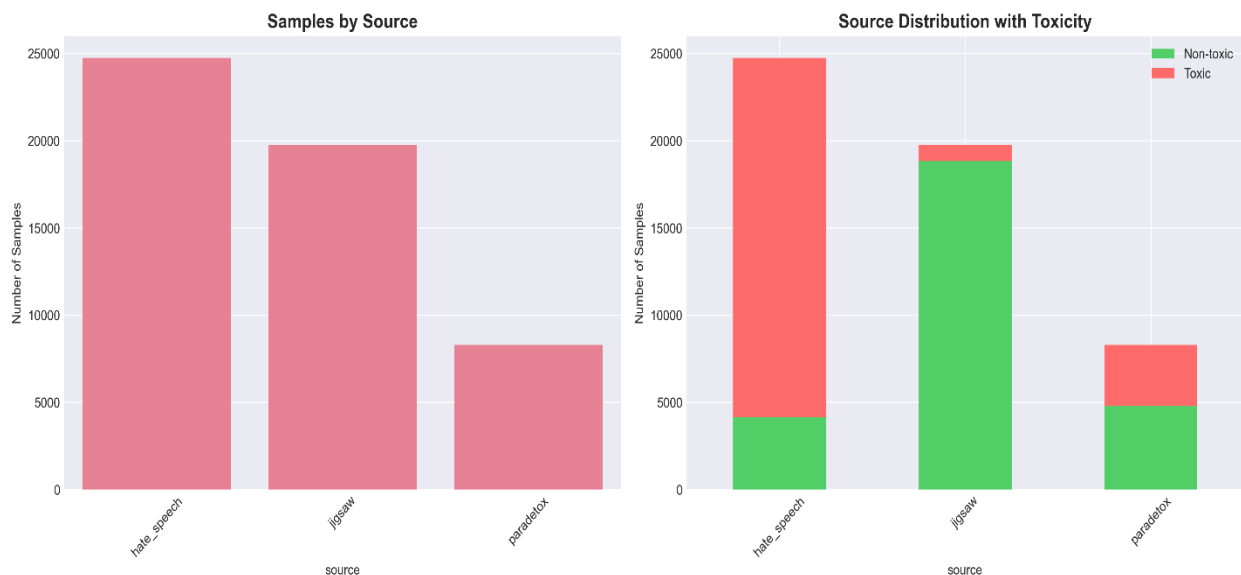


Figure 1: Source Distribution

The Hate Speech dataset represents the largest segment of the raw corpus, around 25,000 samples as shown in Figure 1 (source_distribution). Next is Jigsaw at 20,000 samples, then ParaDetox at roughly 8,200 samples.

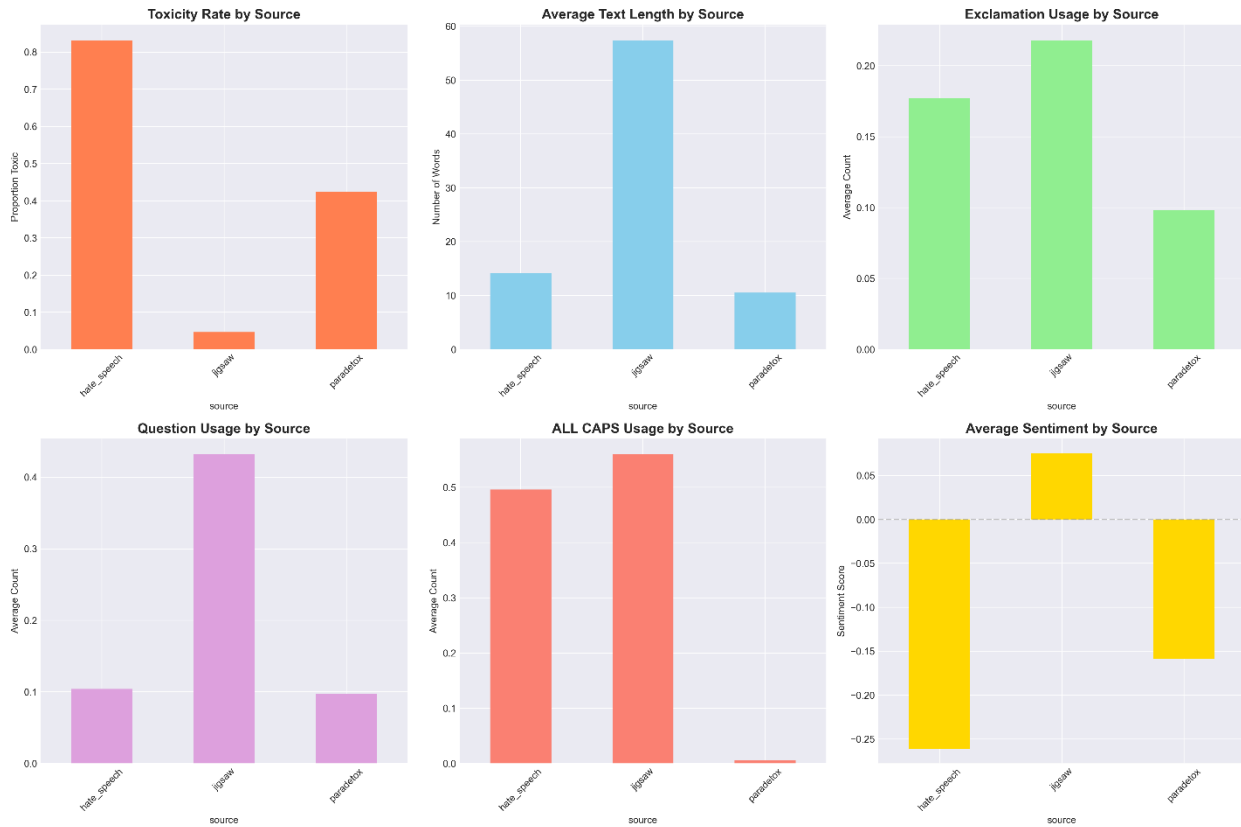


Figure 2: Source Comparison

As Figure 2 (source_comparison) depicts, there is a huge discrepancy in the distributional characteristics of these sources. The Hate Speech dataset has short text comments, with an average of 14 words, high density of toxic language and a highly negative average sentiment (0.25 compound). The Jigsaw comments, on the other hand, are much longer on average (57 words), use standard punctuation (in terms of characters used) and have a slightly positive average sentiment. Disparities in the sources makes a good model rely on a mix of datasets, to learn good detection on various types of texts and language styles.

3.1.2 Preprocessing Pipeline

A schema with two main features was created for all sources: text (the raw comment) and toxic (0 for non-toxic, 1 for toxic). A third feature, source, was added for data provenance in the exploration stage.

Then, a combination and cleaning of the data was performed using `combineandclean.py`. The cleaning process included: removing duplicate entries based on the text field, removing rows where the text or label fields are empty, and removing comments less than 10 characters to avoid an abundance of empty and low information texts. The processed dataset has 52,768 samples,

composed of 25,011 toxic comments (47.4%) and 27,757 non-toxic comments (52.6%), as can be seen in Figure 3.3 (class_distribution.png).

Class imbalance was corrected by stratification balancing the dataset using `traintestdatasets.py`, with the class minority (toxic, 25,011 samples) setting the rate of sampling and the number of samples to choose in the class majority (non-toxic) without replacement. The final balanced dataset has 50,022 samples and was shuffled.

A stratified 80/20 train-test split was then performed. This yields 40,017 training samples and 10,005 test samples, keeping the class balance in both subsets constant. The split used across the different experiments is always this fixed split.

3.2 Exploratory Data Analysis

3.2.1 Text Length Analysis

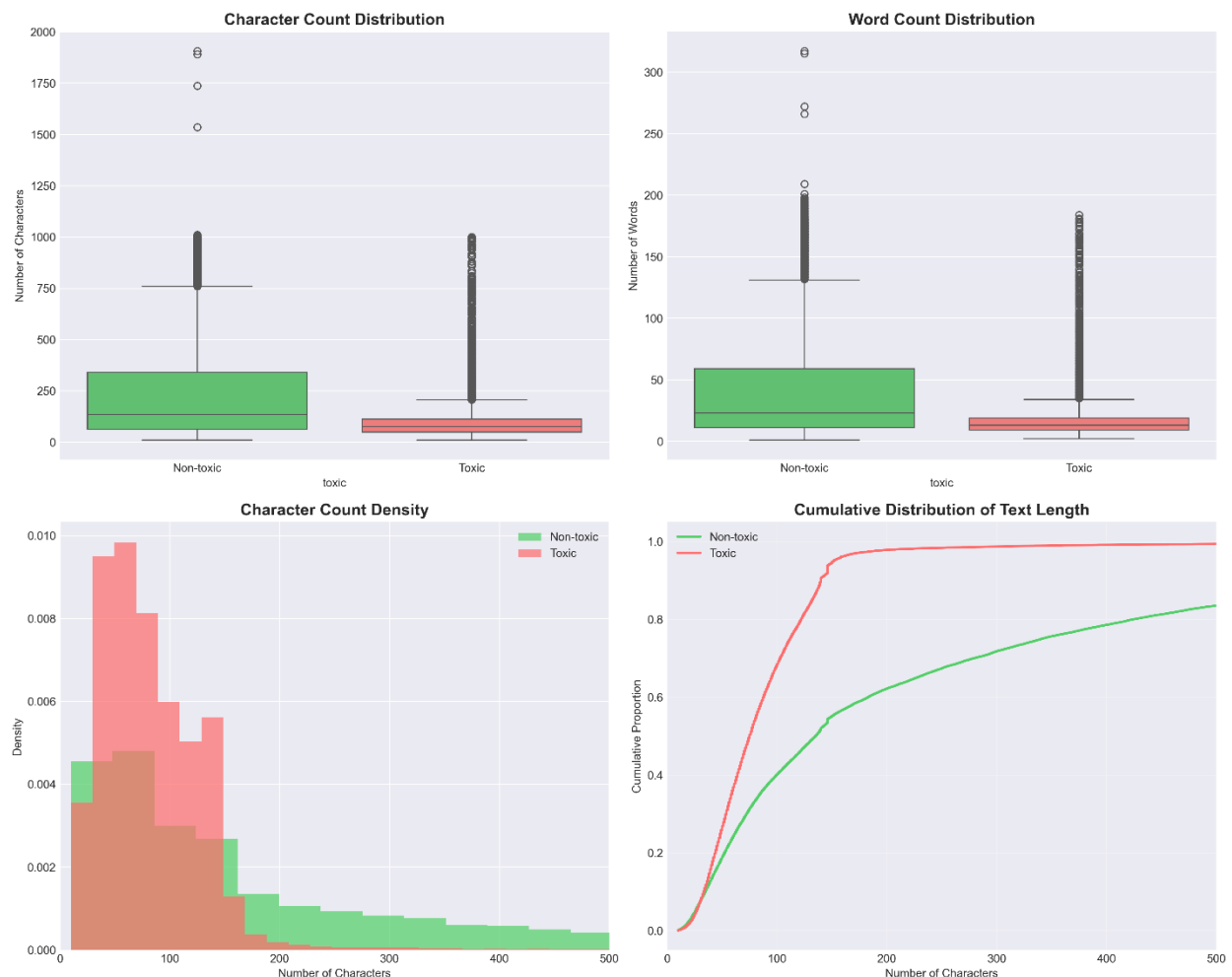


Figure 3: Length Analysis

The most obvious difference between the toxic and non-toxic comments in the combined dataset is their length. The character count distribution for the toxic comments is right-skewed, centered around a median of ~ 70 , whereas non-toxic comments had a much wider distribution centered around a median of ~ 180 (Figure 3: length_analysis). A similar trend is observed in the word count (toxic, ~ 12 words; non-toxic, ~ 30 words), and the cumulative distribution reinforces the difference, showing $\sim 90\%$ of toxic comments are below 200 characters while $\sim 60\%$ of non-toxic comments fall below this length. This difference was observed in all three sources and intuitive in that toxic comments were shorter and more direct, whereas non-toxic ones could be more nuanced.

3.2.2 Punctuation and Capitalization Analysis

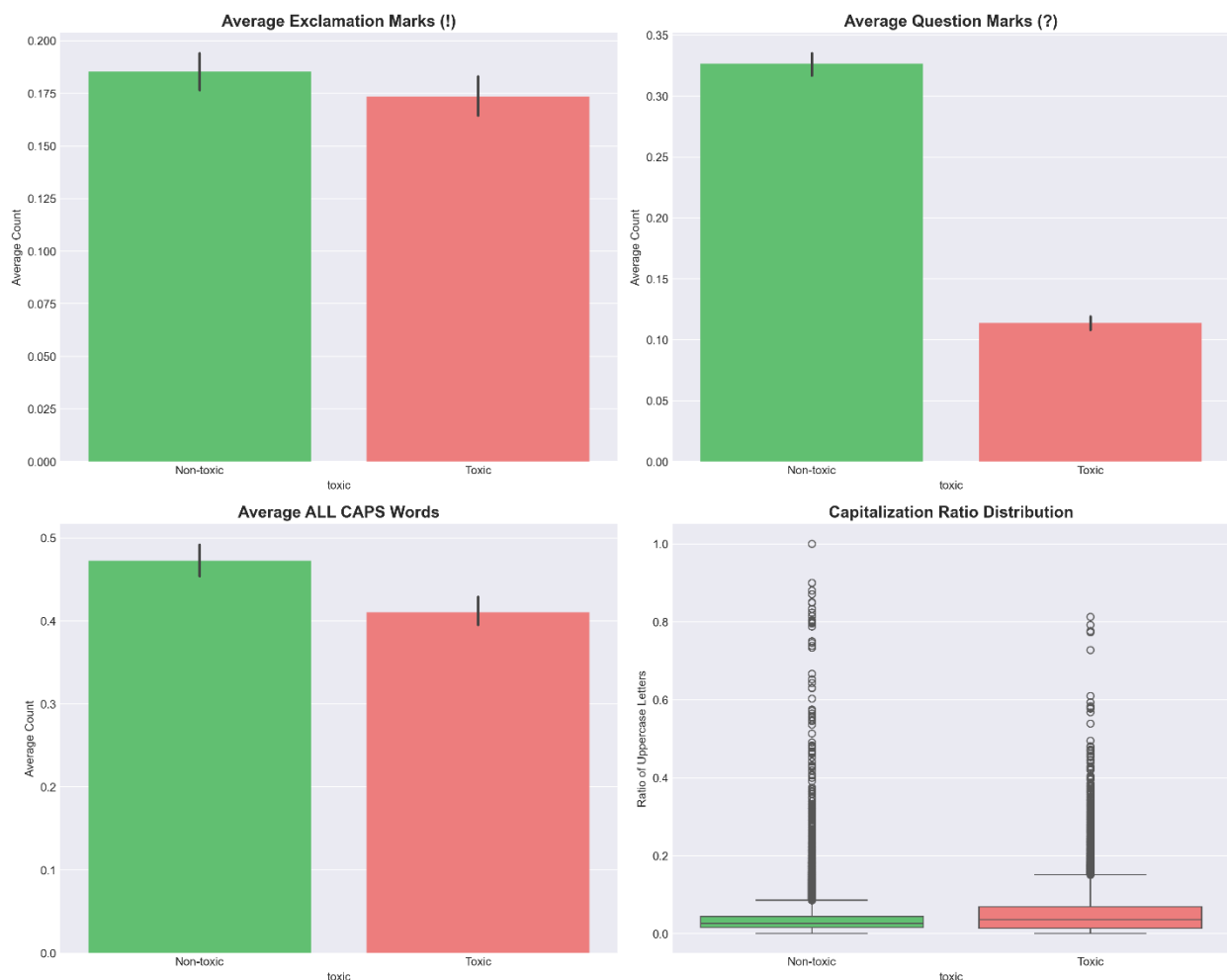


Figure 4: Punctuation Analysis

Figure 4 (punctuation_analysis) compares usage across the two classes. Against common assumptions, non-toxic comments actually average more exclamation marks than toxic comments (0.185 vs. 0.173), but the gap is small. A much larger difference exists for question marks: non-toxic comments contain more (0.33 vs. 0.11), indicating that their speech tends to be more interrogative. All-caps words are marginally more common in non-toxic comments (0.47 vs. 0.41);

given their length, this can be attributed to less emphasis through caps in toxic comments, despite a higher capitalization ratio (ratio of uppercase letters to total letters) overall in toxic ones.

3.2.3 Sentiment Analysis

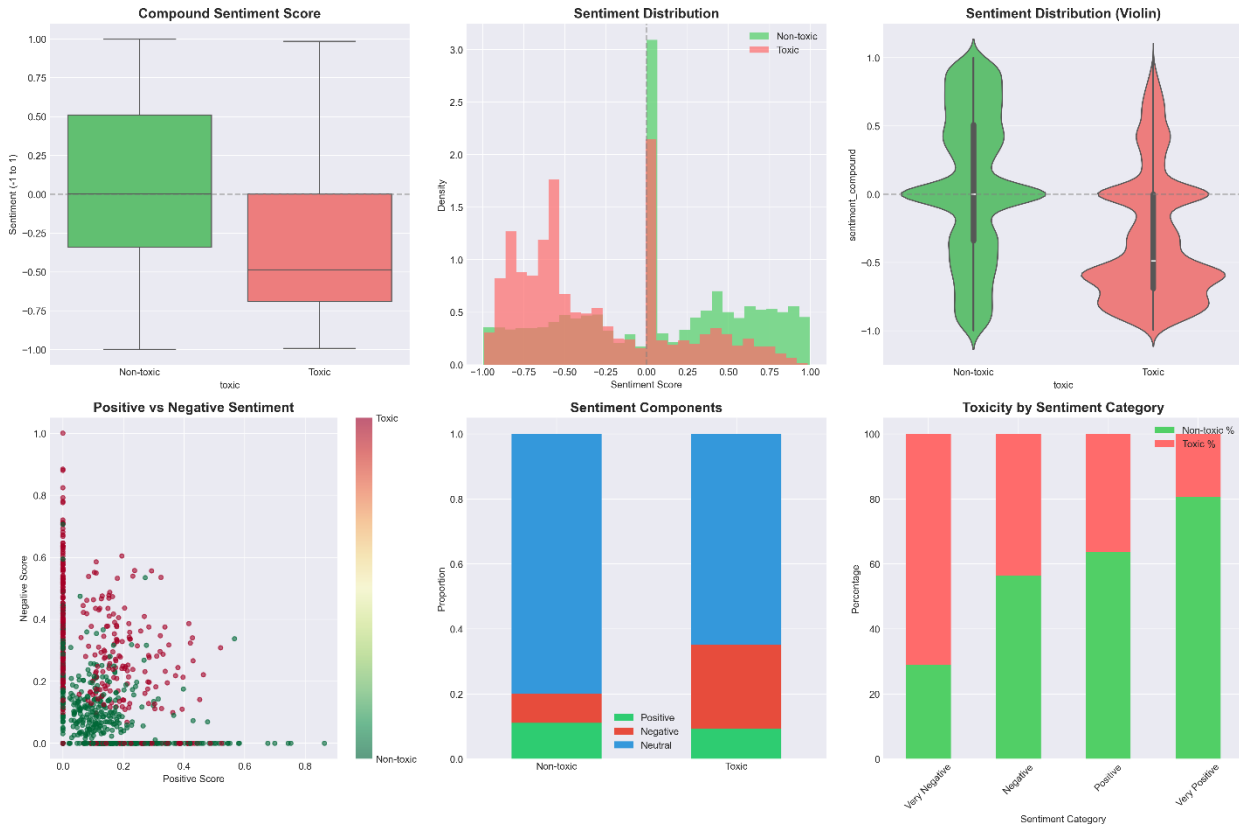


Figure 5: Sentiment Analysis

The VADER (Valence Aware Dictionary and sEntiment Reasoner) lexicon was used to determine the sentiment scores for each comment. As Figure 5 (sentiment_analysis) suggests, the distribution of compound sentiment scores clearly separates the two classes. The toxic comment scores are heavily right-skewed with a median around 0.5 and generally range from negative to moderately positive. The non-toxic comments have a much more symmetric distribution around a median of ~ 0 , ranging from highly negative to highly positive, and are much wider. This separation is confirmed with violins, a positive-vs-negative sentiment plot showing toxicity clustered at highly negative levels, and the component analysis below. Non-toxic comments were predominantly composed of neutral sentiment ($\sim 65\%$) with some negative ($\sim 20\%$) and positive ($\sim 15\%$). Toxic comments had similar proportion of neutral ($\sim 60\%$) but considerably less positive ($\sim 5\%$) and more negative ($\sim 35\%$). This reveals the strength of sentiment and toxicity: highly negative comments had an $\sim 72\%$ probability of being classified as toxic while highly positive comments had an $\sim 20\%$ probability.

3.2.4 Readability Analysis

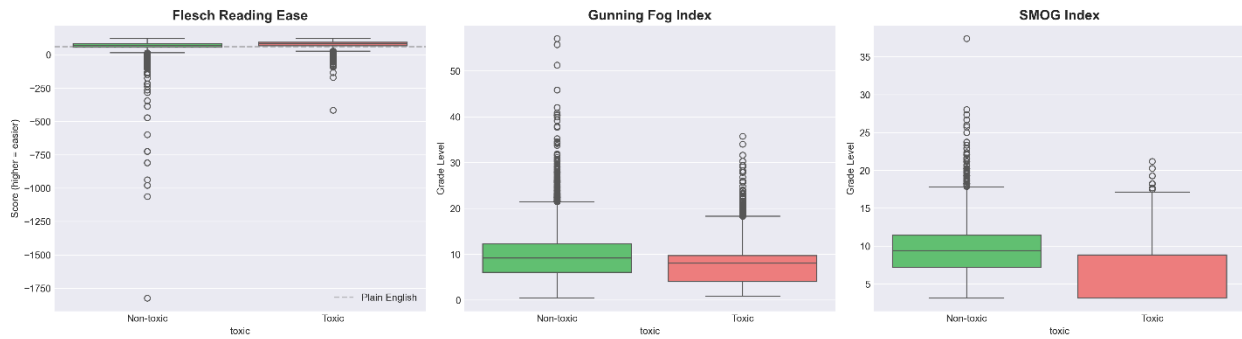


Figure 6: Readability Analysis

Flesch Reading Ease, Gunning Fog, and SMOG are common readability metrics presented in Figure 6 (readability analysis). These reveal that toxic comments have slightly lower Fog and SMOG values and higher Flesch reading ease, but both types cluster mostly in the "plain English" range for Flesch scores. Non-toxic comments had overall higher Fog and SMOG values than toxic ones, indicating a slightly higher linguistic complexity in that group; this aligns with the observation that their sentence structures are somewhat more elaborate and their text lengths are longer.

3.2.5 Lexical Analysis



Figure 7: Wordclouds for Toxic and Non-Toxic Comments

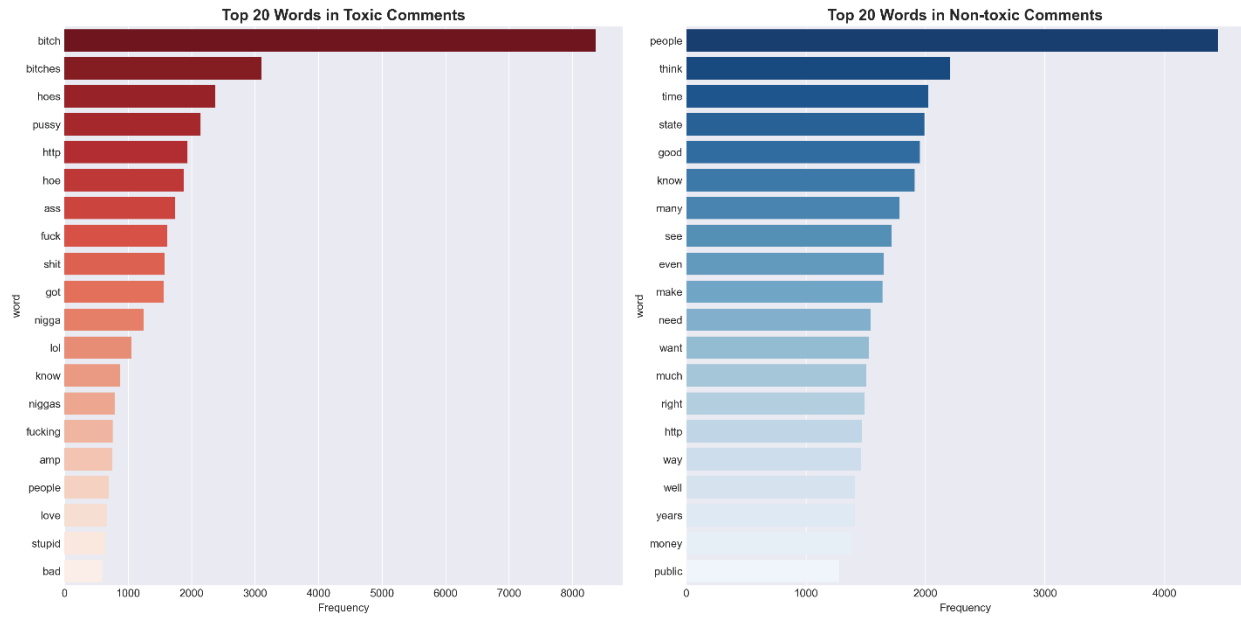


Figure 8: Top 20 words used in Toxic and Non-Toxic Comments

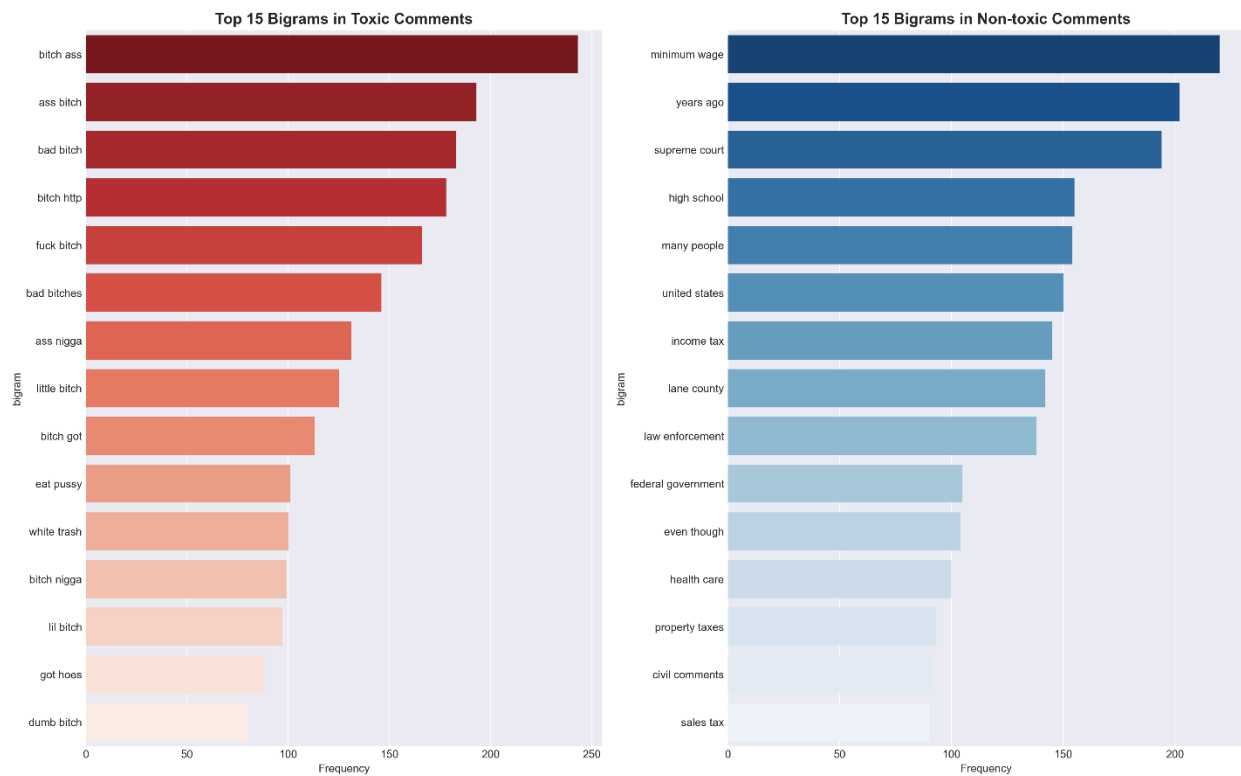


Figure 9: Top 15 bigrams in Toxic and Non-Toxic Comments

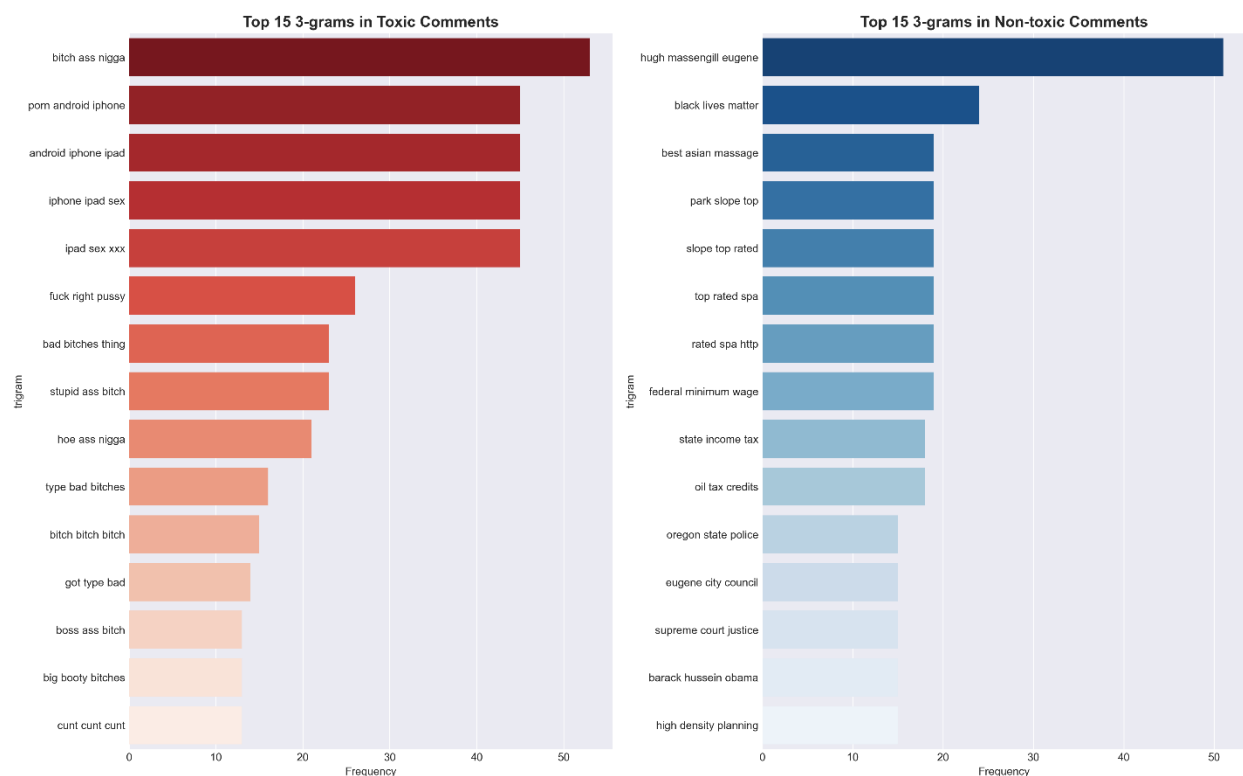


Figure 10: Top 15 3-grams used in Toxic and Non-Toxic Comments

Word clouds and frequency-based plots are used in Figures 7, 8, 9, 10 (wordclouds, top_words, bigrams, 3-grams). The toxic word cloud is overwhelmingly centered around profanity and slurs, the most frequent of which has appeared >8400 times. The non-toxic word cloud reflects more standard topic-driven discourse (people, think, time, state, good are common words and reveal the prevalence of news data), and also appears considerably more dilute. When looking at common bigrams and trigrams the trends are continued. Most frequent toxic trigrams are pairs/groups of insults and highly negative words. Most frequent non-toxic trigrams represent topic areas (e.g. "minimum wage", "supreme court", "state income tax"). This implies that models relying on topic words alone may have success at standard cases, but are prone to failure if these words can be replaced or other wording is used.

3.3 Feature Engineering and Correlation-Based Selection

3.3.1 Engineered Features

A total of fifteen scalar features are created per comment for input into the classical ML models. The list below details the ten features inspired by the linguistic features analyzed above, separated into 5 categories as presented in the following table.

Category	Features	Rationale
Text length	char_count, word_count, sentence_count	Toxic comments are much shorter (median ~70 chars compared with ~180 chars for non-toxic).
Punctuation	exclamation_count, question_count, period_count	Punctuation trends vary between aggressive and discursive discourse style.

Capitalisation	all_caps_words, caps_ratio	Aggressive "shouting" sometimes results in over-capitalisation of comment text.
Sentiment (VADER)	sentiment_compound, sentiment_positive, sentiment_negative, sentiment_neutral	Negativity of sentiment was the best single predictor of comment toxicity ($r = 0.46$ Pearson).
Readability	flesch_score, fog_index, smog_index	The non-toxic civil-comment text tends to be more sophisticated linguistically than the shorter "social media insult" commentary.

Sentiment features were calculated using the VADER lexicon, which is well-suited to social media text as it accounts for capitalization, punctuation and casual writing. Readability scores were calculated using the textstat library, implementing formulas such as Flesch Reading Ease, Gunning Fog index, and SMOG index. Prior to training, features were normalized to mean 0 and unit variance.

3.3.2 Correlation analysis

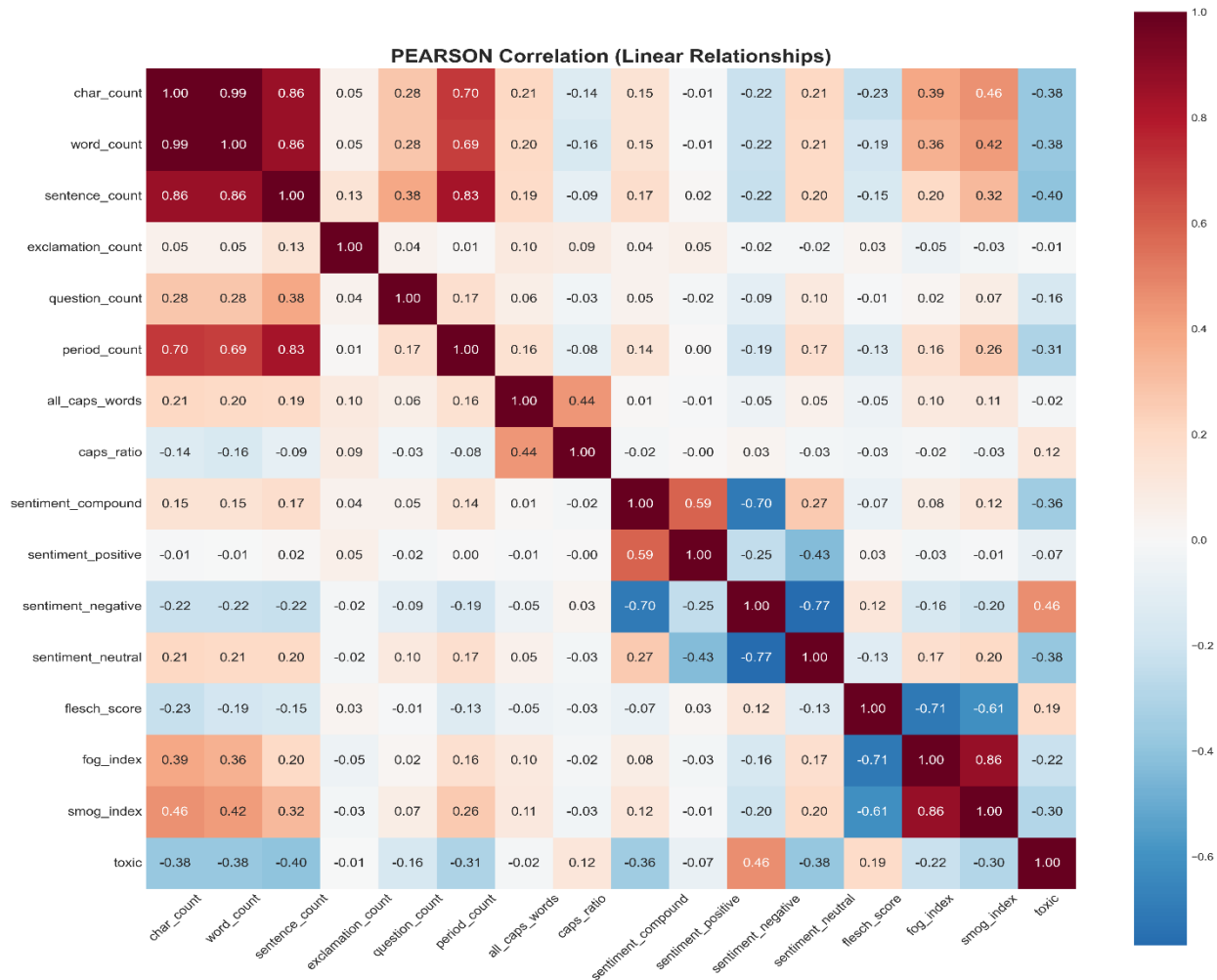


Figure 11: Pearson Correlation Matrix

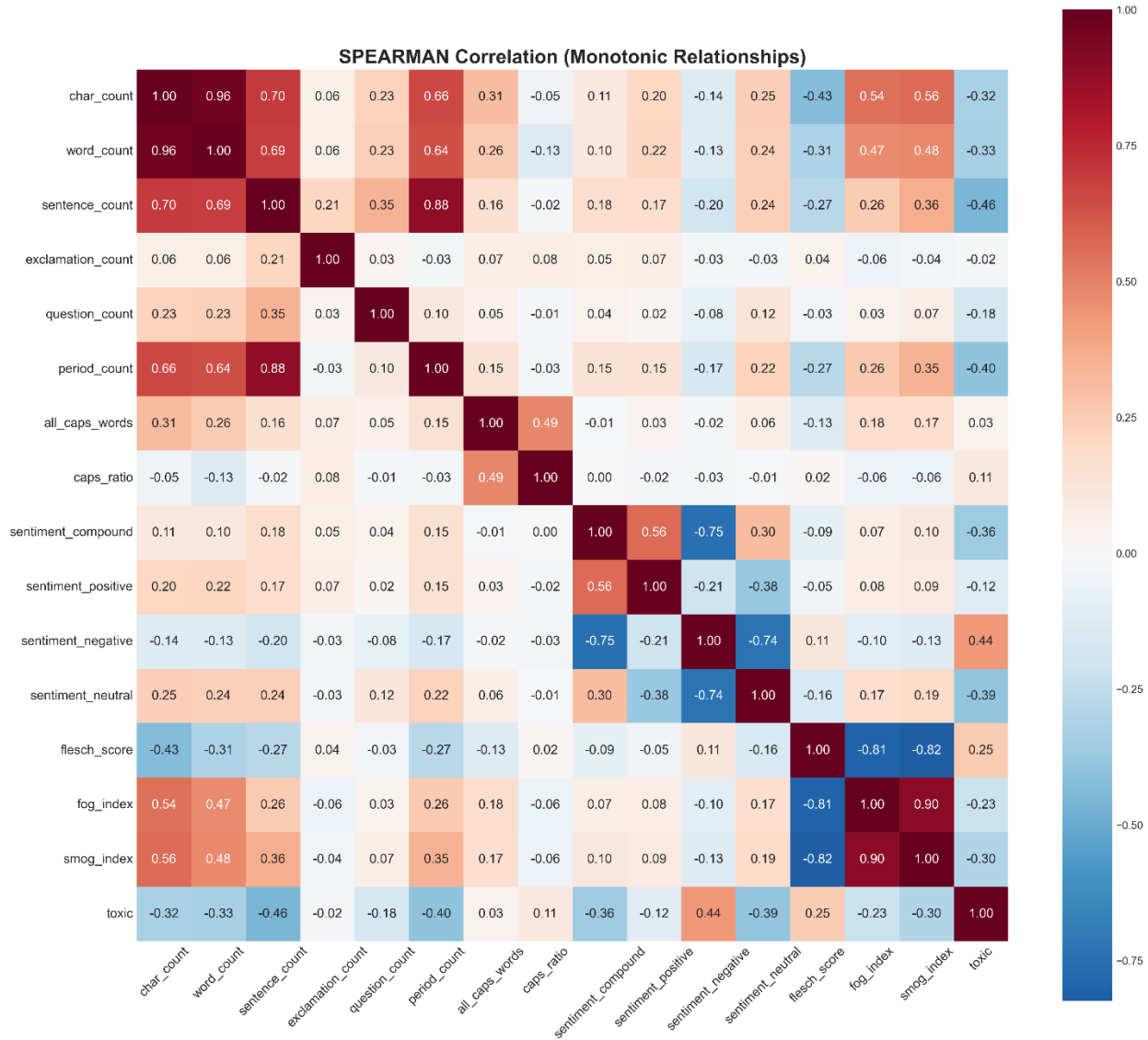


Figure 12: Spearman Correlation Matrix

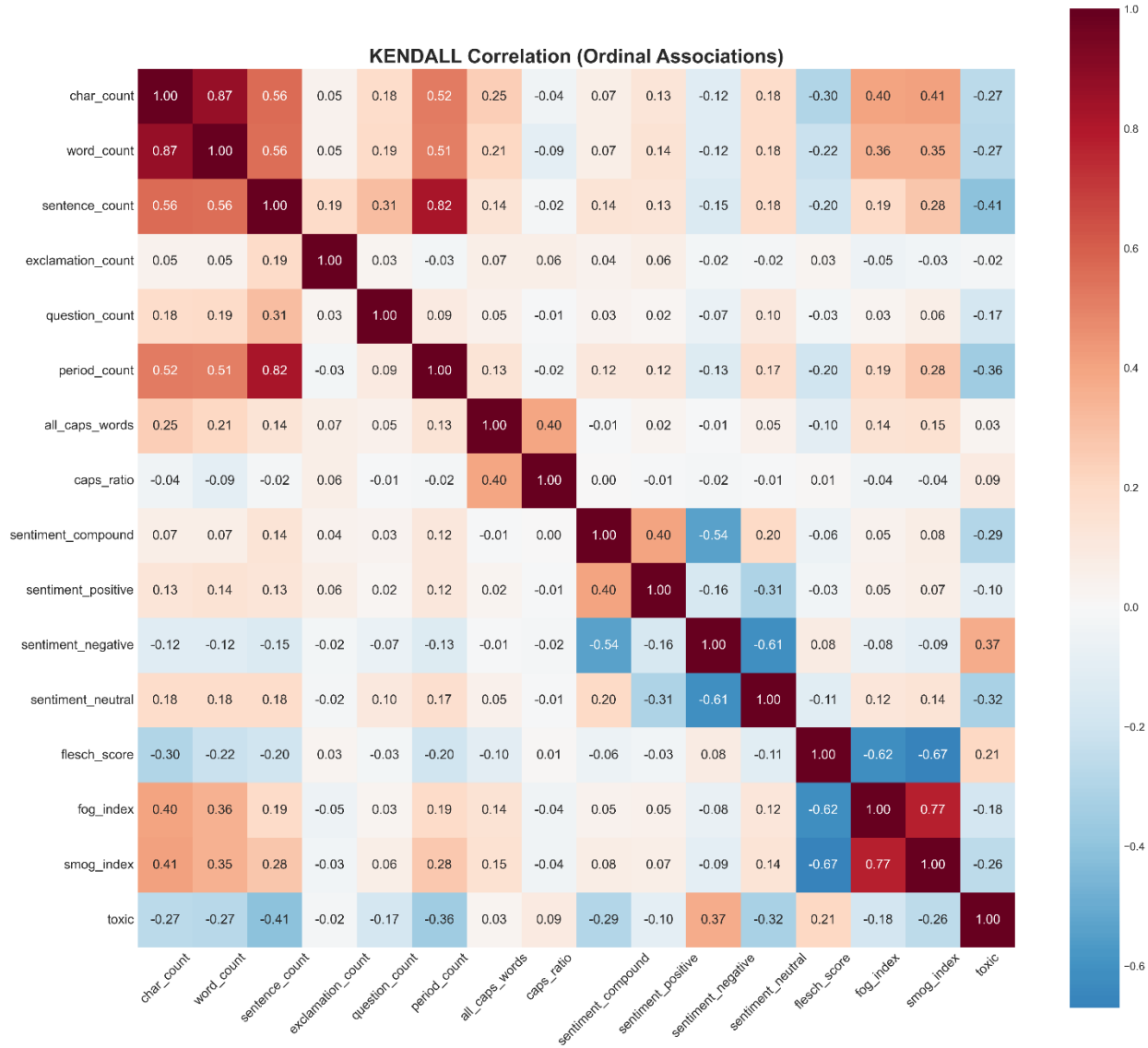


Figure 13: Kendall Correlation Matrix

The correlation between the 15 engineered features and the binary toxicity label were investigated using three correlation measures; Pearson correlation to assess linearity (Figure 11), Spearman rank correlation to assess monotonicity (Figure 12), and Kendall's tau to assess ordinal dependence (Figure 13). The use of multiple measures of correlation characterize the relationships better, as Pearson assumes linearity and is susceptible to outliers, while Spearman and Kendall do not assume a relationship and are better suited for zero inflated and skewed features like punctuation count.

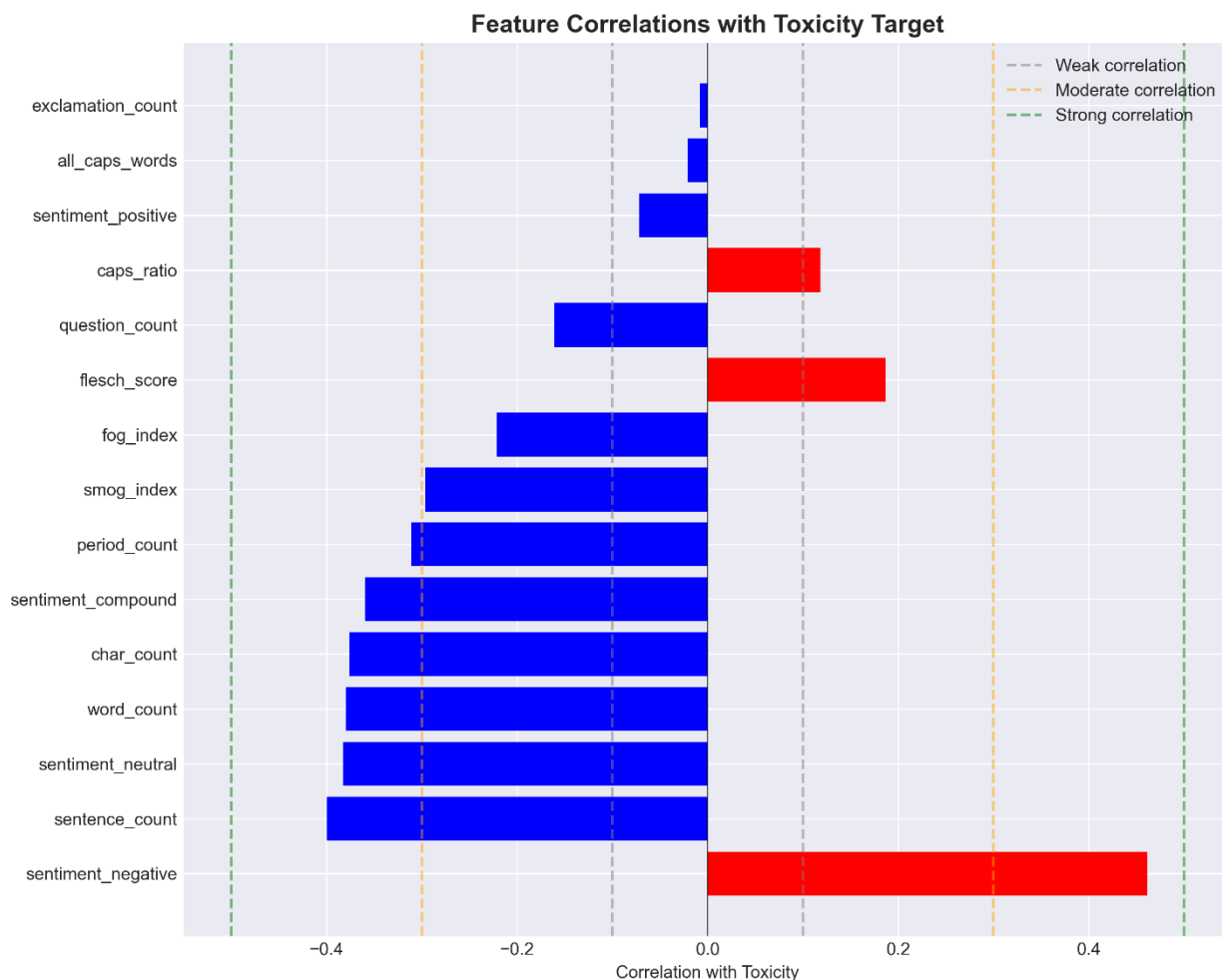


Figure 14: Visualization of Features correlation with Target in Particular

The correlation values between features and the target labels, visualized in (Figure 14, target_correlations), confirm that sentiment negativity is the most significant feature, with Pearson correlation $r = 0.46$, the only one above the moderate correlation threshold of 0.30. The trend remained consistent across all metrics (Spearman = 0.44, Kendall = 0.37). Text-length features follow, all having moderate-to-weak negative correlation with the target, including sentence count ($r = 0.40$), char count ($r=0.38$), word count ($r=0.38$), and period count ($r=0.31$) which reflect shorter toxic text lengths seen during EDA. Readability related scores had similar moderate to weak negative correlation with the target including SMOG index ($r = 0.30$) and Fog index ($r=0.22$) as opposed to positive correlation with the target for the Flesch score ($r=0.19$), as expected with simpler text features being more prevalent. Capitalization related features and exclamation marks were all weakly to negatively correlated with the target (near zero); this contradicts our intuition of them being associated with stronger malicious content. This result is significant when determining features that may not be as valuable, as even if they have intuitive merit, their descriptive power is not strong enough. All 15 features are still kept for the classical models to allow any potential non-linear feature interaction to be found; and for interpretability of the correlation analysis.

3.3.3 Random Forest Feature importance

An additional perspective on the 15 features can be found by examining the Random Forest model's permutation feature importance scores (as shown in Figure 3.16). The most influential features identified were; sentiment\negative (8.95%), most frequent offensive unigram (8.18%), char\count (7.83%), word\count (7.32%), and sentiment\nneutral (6.44%). The overall top features closely align with the correlation values, and are similarly dominated by sentiment and text-length features. Punctuation based features remain close to 0 with exclamation_count being ranked 123rd overall and having a score of only 0.055%. The inclusion of offensive unigram as a feature explains some of the gap between the Random Forest score and other models such as the SVM trained on only featureengineered variables as it specifically addresses lexically toxic phrases. All 15 features are included for the purpose of understanding how much influence, however minute, they have on the classifier and to show they don't detract from performance for non-linear methods.

3.4 Baseline model - Logistic Regression (TF-IDF)

A Logistic Regression classifier trained on TF-IDF representations served as the base for this work. The vectorizer was setup to allow a vocabulary size of up to 50,000 words and accept n-grams between (1, 3). The classifier was trained with a class weight balanced for each class, maximum 1000 iterations, and C = 1.0. Using the withheld 10,005 item test set the classifier produced: Accuracy: 92.37%, Precision: 0.887 (non-toxic) 0.969 (toxic), Recall: 0.972 (non-toxic) 0.876 (toxic) Macro F1: 0.924. A small deviation between precision and recall for toxic indicates that this classifier is less likely to correctly identify subtle toxins and makes more false negative errors; this can be attributed to the TF-IDF feature selection, which heavily relies on direct lexical features. However the benchmark that this baseline model sets is considerable and is attributable to the clearly separated toxic comments in terms of lexicon, as seen during EDA.

3.5 Optimised classical models - SVM, KNN and Random Forest

The three classical models below were tested with randomized grid search, using 5-fold stratified cross-validation to pick best parameters based on cross validated F1 score of the 15 engineered features. This method ensures a fair comparison against the TF-IDF benchmark.

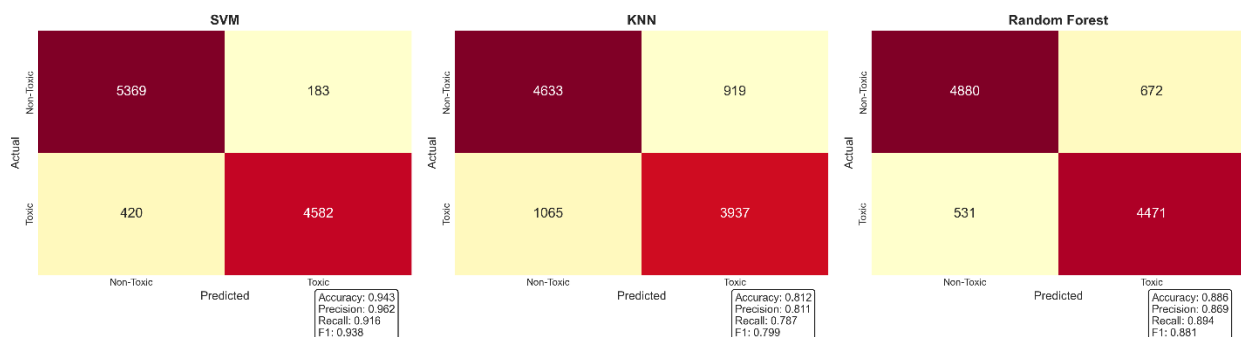


Figure 15: Confusion Matrices for the Three Optimized Models

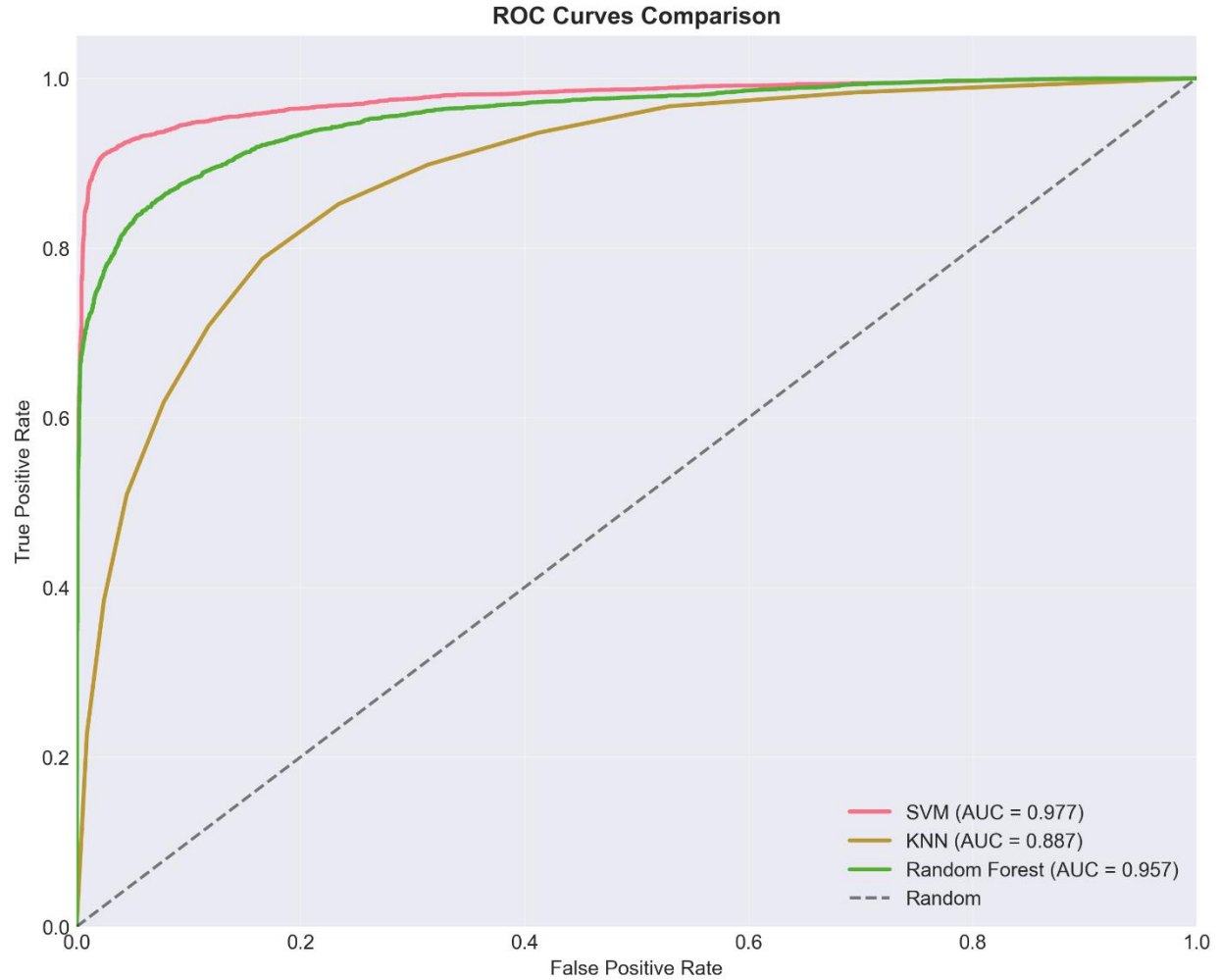


Figure 16: ROC Curves Comparison between the Classical Models

3.5.1 SVM

The parameters optimized for the SVM were the kernel type (linear, RBF, poly), C, and gamma value. The linear kernel was the best fit with $C=3.234$, $\gamma=0.098$, producing a cross validated F1 score of 0.9348. With the test set the model yielded Accuracy: 94.29%, Precision: 0.962, Recall: 0.916, F1-score: 0.938, AUC: 0.977. The confusion matrix (as shown in Figure 15) shows the high accuracy, with low false positives and false negatives indicating the engineered features are very discriminative when analyzed with the linear kernel; this model outperforms all previously evaluated methods.

3.5.2 KNN

The parameters that were chosen for the KNN classifier were number of neighbours (k), distance metric, weighting strategy and algorithm. The model chosen for this test case was $k=11$, Euclidean distance, with uniform weights and the ball tree search algorithm. The test set yielded: Accuracy: 81.20% F1: 0.799 AUC: 0.887. The KNN classifier performed worst of the classical models. This is attributed to its dependency on local relationships within the feature space, and toxic comments

that are structurally distant but contextually toxic are not recognized by KNN as well as other classifiers.

3.5.3 Random Forest

The optimized Random Forest model included the following parameters: number of trees (169), maximum depth (30), and split criterion. The results for the test set were: Accuracy: 88.60% F1: 0.881 AUC: 0.957. This classifier produced good recall. Overall ROC comparisons clearly show the order SVM>RF>KNN.

3.6 ToxiGuard Architecture

The ToxiGuard architecture consists of two modules that work together to perform classification and provide explanations, respectively. The classifier is based on fine-tuned BERT, whereas the explanation generation is powered by a fine-tuned GPT-2. They are trained independently, and only at runtime when a prediction is made, if the comments are classified as toxic then the GPT-2 is invoked to give an explanation.

3.6.1 BERT Fine-tuning (Classification Head)

A fine-tuned version of BERT-base-uncased (12 layers, 768 hidden sizes, ~110 million parameters) is used as the classification head, with an added linear classifier attached to the [CLS] token representation. Learning rate was 2×10^{-5} batch size 32, for 3 epochs with a maximum sequence length of 128. When run on the 10,554 item test set the model gave: Accuracy: 96.07%, Precision: 0.961, Recall: 0.957, Macro F1: 0.960. There is little imbalance between precision and recall, indicating fairly balanced errors made by the classifier. The inherent power of contextualized word embeddings allows BERT to recognize subtler forms of toxicity when compared to TF-IDF based classifiers.

3.6.2 GPT-2 Fine-tuning (Explanation and Rewriting Head)

The generative model is based on a fine-tuned version of GPT-2 (117 million parameters) to perform a specified task through the generation of text. In this case, an input of the toxic comments and a prompt structure were provided, forcing the generation of structured text which includes; 1) an explanation as to why the text is deemed toxic, and 2) 3 alternative constructions that still achieve the purpose of the original text but without the offensive nature.

Field	Content
Comment	[input toxic comment text]
Label	TOXIC
Explanation	This comment is toxic because [reason]. [Impact statement].
Alternative	Consider saying instead: [constructive rewrite of the core message].

To fine tune the GPT-2 model, pairs of toxic comment, corresponding explanations drawn from the training set. These explanations and the alternate rewrites are generated by means of a category aware annotation where categories of toxicity, such as threatening, highly insulting, and demeaning were mapped to particular explanation templates. The directed supervision requires the model to provide consistent, contextual, category-specific responses. At inference time, the system

feeds the toxic comment to the model and autoregressively generates the remaining part of the structured output with greedy decoding.

Two samples drawn from the held-out test set further exhibit the behavior of the generative component. Given toxic comment "RT @DeePaPi1800: That outside dick keep dem hoes sick..." (toxicity confirmed with BERT score 0.996), an explanation is provided that states that offensive content is detected and suggests alternatives such as being frustrated but not hostile:

For comments consisting of harsh but non-threatening statements a different template is provided that points out that the comment is overly hostile and hinders constructive dialogue, and provides alternative phrasings that do not deviate from the intention of the statement.

3.6.3 Combined ToxiGuard Inference Pipeline

The input comment is first passed to the BERT classification head to predict a binary toxic/non-toxic label along with the probability score. If the comment is labeled as non-toxic, the system outputs the confirmation message (e.g. Comment is safe) and finishes the task. If it's labeled as toxic it is passed on to the GPT-2 generative module to generate a structured response which comprises of a natural-language explanation and three constructive alternatives. The final output to the system is thus the classification label, the confidence score, the explanation, and suggested alternatives: It may be noted that by virtue of the modular design, the time-critical classification component is independent from the generative component, so that it may be updated, tuned or substituted separately. Furthermore, it may also be used for more flexible applications, for example confidence based thresholding could be employed to decide how much of the problematic comments to explain.

Model	Accuracy	Precision	Recall	F1-score	AUC
Logistic Regression (baseline)	92.37%	0.928	0.924	0.924	—
KNN (engineered features)	81.20%	0.811	0.787	0.799	0.887
Random Forest (engineered features)	88.60%	0.869	0.894	0.881	0.957
SVM (engineered features)	94.29%	0.962	0.916	0.938	0.977
BERT (fine-tuned)	96.07%	0.961	0.957	0.960	—
ToxiGuard (BERT + GPT-2)	96.07%	0.961	0.957	0.960	—

The step up from the TF-IDF Logistic Regression baseline (92.37%) to the classical models built with engineered features (81.20-94.29%), and finally to the fine-tuned BERT classifier (96.07%), demonstrate the improvement in performance through ever more representational text models.

Remarkably, the SVM trained on engineered features reaches 94.29% accuracy, second only to BERT at 96.07%, which indicate that the 15 handcrafted features extract much of the inherent discriminative power of the data. Nonetheless, the difference between their accuracy scores is a reflection of how BERT’s word-level, context dependent features can exploit aspects of the data inaccessible to hand-crafted surface features.

3.7 Code Availability

All code implementing ToxiGuard, including scripts used for data collection, preprocessing, exploration, feature engineering, feature correlation, classical models optimization and BERT and GPT-2 fine-tuning pipeline, is freely available under the following Github repository:

<https://github.com/aminehabbou/ToxiGuard>

For storage reasons, neither the fine-tuned weights for the BERT classifier, nor the fine-tuned weights for the GPT-2 generator, nor the final trained ToxiGuard model, nor the processed datasets are present in the repository, because they are very large files to be committed. But they are fully reproducible using the instructions given in the README file.

4. Experiments and Results

4.1. Experimental Configuration

All experiments were performed on the balanced dataset described in Section 3.1. To ensure replicability all stochastic processes were performed with a seed set to 42. A 80-20 split was performed on the dataset using stratification, which resulted in 40017 samples for training and 10005 for the held-out test set, with 50/50 split in both sets.

All BERT and full ToxiGuard experiments are tested on a 10554-samples test set. This disparity stems from an anomaly that occurs with specific inputs during tokenization. Thus, all classical models (Logistic Regression, SVM, KNN and Random Forest) are tested on a 10005-samples test set while BERT and ToxiGuard are reported on the 10554-samples test set. This particular point is always maintained and stated in each result table.

The hyperparameter optimization for the classical engineered feature models was done through randomized grid search with 5 fold stratified cross-validation, selecting the hyperparameters based on macro F1-score. For the Logistic Regression baseline, fixed hyperparameter was used, without any search.

The BERT model fine-tuning process uses AdamW as the optimizer, with a learning rate of $2e-10$, a batch size of 32, an epoch number of 3, and a max sequence length of 128. The generative component GPT-2 was fine-tuned using AdamW with a learning rate of $5e-10$ for 2 epoch on the structured template dataset.

All presented classification metrics are averaged using macro averaged for both classes unless stated otherwise, for no specific class is favored against the other.

4.2. Classical Models – Quantitative Results

A confusion matrix of statistics per class can be found below in the table below for each of the four classical models.

Model	Class	TP	TN	FP	FN	Prec.	Recall
Logistic Regression	Non-Toxic	—	—	—	—	0.887	0.972
	Toxic	—	—	—	—	0.969	0.876
SVM	Non-Toxic	5,369	4,582	183	420	0.962	0.928
	Toxic	4,582	5,369	420	183	0.962	0.916
Random Forest	Non-Toxic	4,880	4,471	672	531	0.879	0.879
	Toxic	4,471	4,880	531	672	0.869	0.894
KNN	Non-Toxic	4,633	3,937	919	1,065	0.813	0.834
	Toxic	3,937	4,633	1,065	919	0.787	0.787

The Logistic Regression baseline (50k TF-IDF features with unigrams to trigrams) achieved an accuracy of 92.37% and a macro F1-score of 0.924. However, it has very unbalanced class performances. While its precision for the toxic class is high (0.969), its recall is much lower (0.876), meaning about 1 in 8 toxic comments are missed. This is very typical for TF-IDF based models, they perform very well for clear offensive words, but not for implicitly toxic statements and creative insults.

In terms of the engineered feature models, the Support Vector Machine (SVM) with linear kernel reached the highest performance (accuracy 94.29%, F1-score 0.938, AUC 0.977) on the task. Its confusion matrix shows relatively balanced errors, with 183 false positives and 420 false negatives, and it has the highest F1-score. The success of the linear kernel means that the 15 engineered features (especially sentiment negativity, text length and readability) were almost linearly separable in the feature space and well suited for the large-margin optimization of SVMs.

The Random Forest classifier achieved an accuracy of 88.60%, F1-score of 0.881, AUC of 0.957. Although its overall accuracy was less than the SVM, it showed higher recall (0.894) for the toxic class compared to the other three models. It also achieved 672 false positives which are more than SVM. In situations when mistakingly classified toxic for non-toxic has higher costs than mistakingly classified non-toxic for toxic, Random forest might be a good option. Feature importance shows that sentiment negativity (0.090), character count (0.078) and word count (0.073) are the top 3 structural features while highest frequent offense word (top 1 unigram, 0.082) is also a very important structural feature. It seems the Random Forest relied on both the structural and lexical features.

K-Nearest Neighbors (KNN) performed the worst in terms of all measures (accuracy 81.20%, F1-score 0.799, AUC 0.887). Its confusion matrix also showed that it performed badly on both direction with 1,065 false negatives and 919 false positives. The optimized performance (k=11, Euclidean distance, ball-tree) with cross-validation had a F1-score of 0.795. Its bad performance is probably because the KNN model heavily relied on local data distribution; for comments with rare combinations of feature values, such as a very long, grammatically perfect, yet toxic statement, similar neighbours cannot be easily found in the training set.

Comparison on ROC curves (Figure 3.17) confirmed the relative performance ordering, where SVM (AUC 0.977) > Random Forest (AUC 0.957) > KNN (AUC 0.887).

4.3 ToxiGuard - Quantitative results

The per-class classification report of the BERT component on the total test set of 10,554 examples is displayed in the table below:

Class	Precision	Recall	F1-score	Support
Non-Toxic	0.9616	0.9642	0.9629	5,552
Toxic	0.9601	0.9572	0.9587	5,002
Macro Average	0.9608	0.9607	0.9608	10,554

The fine-tuned BERT classifier yielded an accuracy of 96.09% and a macro F1 score of 0.9608, representing a substantial improvement over all traditional models. Class performances are almost equally balanced, with non-toxic: (Precision=0.9616, Recall=0.9642, F1=0.9629) and toxic: (Precision=0.9601, Recall=0.9572, F1=0.9587). This symmetry is attributed to both a balanced training data and the ability of BERT to capture context and model relational dependencies allowing for toxic content not contained within an explicit keyword.

The confusion matrix is shown in Figure 3.18, and consists of 5353 true non-toxic, 4788 true toxic, 199 false positives, and 214 false negatives. Overall, 413 out of 10,554 were misclassified and as this number is very small relative to the entire test dataset, we see that there are $199+214 = 413$ misclassifications. Given how similar the FP and FN values are, the system is behaving in a well calibrated way.

When inspecting the false negatives and false positives, we note that in most cases, a relatively confident prediction near the classification boundary is responsible for the error. The average prediction confidence of a false negative is 0.778 (stdev=0.151) and the average confidence for a false positive is 0.730 (stdev=0.153). It appears that few highly confident predictions are wrongly made and that most mistakes occur when the prediction confidence is less extreme. Manual inspection of these misclassified examples provided three main categories of difficult cases. 1. Diluted toxicity: where some mild toxicity is buried within a comment that otherwise expresses no offensive opinion or emotion. 2. Figurative context, such as irony or sarcasm; and 3. Inputs that are themselves very short, contain very little context, and hence lead to inherently uncertain model predictions.

In the pipeline system (ToxiGuard), the generative part using GPT-2 was called for all of the 4987 inputs classified by the BERT classifier as toxic. In every case, a concise explanation for the classification, and three constructive ways to rewrite the original input in a non-toxic way, were returned. All generative calls returned successfully with no errors, displaying robust operation of the integrated inference pipeline.

4.4 Model Comparison

The table below compares all four models. As we move across the row, the performance steadily increases from simpler models to the entire ToxiGuard system.

Model	Test n	Accuracy	Precision	Recall	F1	AUC
Logistic Regression (TF-IDF)	10,005	92.37%	0.928	0.924	0.924	—
KNN (engineered features)	10,005	81.20%	0.811	0.787	0.799	0.887
Random Forest (engineered features)	10,005	88.60%	0.869	0.894	0.881	0.957
SVM (engineered features)	10,005	94.29%	0.962	0.916	0.938	0.977
BERT fine-tuned	10,554	96.09%	0.961	0.961	0.961	—
ToxiGuard (BERT + GPT-2)	10,554	96.09%	0.961	0.961	0.961	—

The transition from the Logistic Regression baseline (92.37%) to engineered-feature models (81.20-94.29%) and eventually to a fine-tuned BERT classifier (96.09%) is demonstrative of how each improvement in representational power correlates with better performance. Among classical approaches, SVM is the best model at 94.29% accuracy, effectively closing most of the gap between classical approaches and BERT; it falls short by only 1.80 percentage points. This is a remarkable result: the SVM is achieved on the back of a very small 15 engineered features, takes seconds to train, and achieves accuracy only moderately below the 110M parameter BERT. This likely stems from the very good linear separability found between feature classes during our correlation analysis, in which a number of sentiment-linked features are among the strongest correlates of toxicity.

The 1.80% accuracy gap is nevertheless quite significant. Bringing error rate from 5.71% (SVM) to 3.91% (BERT) is a ~32% reduction in the error rate, or around 180 fewer comments classified wrongly out of every 10,000. This reduction is almost exclusively concentrated on the harder, context-dependent classes such as indirect threats and toxicity whose definition depends heavily on situation or ironically positive wording, and on which simple features do not succeed but the contextual embeddings excel.

This is also evident from the relatively poor performance of the KNN model (81.20%), which showed that neighbor-based algorithms did not perform well on this feature space even with exhaustive tuning; indeed, models like these must possess the ability to capture global structure and complicated decision boundaries-e.g. Through kernels or ensemble learning-to succeed on such a task.

ToxiGuard inherits the high performance of BERT for classification (96.09% accuracy, macro F1-score of 0.9608), and supplements it with a generative GPT-2 component, without any decrease in detection accuracy. The modular approach of treating classification and generation separately means each task is treated in the most appropriate way.

4.5 Qualitative Analysis and Examples

To provide a richer evaluation than simple quantitative accuracy figures, a few inference examples on the held-out test set are given below. These showcase instances of both correct system performance, as well as common classes of error.

4.5.1 Correct Classifications with GPT-2 Output

The following show ToxiGuard correctly classifying a range of toxic comments, and providing helpful explanatory text and positive rephrasings:

Example 1 — Explicit offensive language (BERT confidence: 0.996)

```
Input:  "RT @DeePaPi1800: That outside dick keep dem hoes sick..."
Label:  Toxic | Prediction: Toxic | Confidence: 99.6%

GPT-2:  it includes threatening language. Wishing harm on others
        creates a
        hostile environment and is never acceptable. Consider
        saying:
        - I'm angry, but I want to address this constructively.
        - This really upset me, and I need to say that clearly.
        - Let's deal with the issue directly instead of escalating
        it.
```

Example 2 — Harsh personal attack (BERT confidence: 0.998)

```
Input:  "That damn lispy mouth bitch knows good in hell Tara wasn't
a
        damn roommate! Damn rocks for brains man!"
Label:  Toxic | Prediction: Toxic | Confidence: 99.8%

GPT-2:  the language here is unnecessarily harsh. Strong words can
        make
        people defensive and stop listening to what you have to
        say.
        Consider saying:
        - I feel strongly about this, and here's my honest view.
        - I'm really frustrated by this situation.
        - I strongly disagree with that position.
```

Example 3 — Non-toxic comment (correctly cleared by pipeline)

```
Input:  "If you're not an elected retiree official, Kearon, sit
down
        and read this medical benefits report for Alaska's elected
        officials and then come back and share your thoughts..."
Label:  Non-Toxic | Prediction: Non-Toxic | GPT-2: not invoked
Pipeline output: "Comment is safe."
```

4.5.2 Failure Cases

We analyze two example error types below, in an attempt to characterize the model's 3.91% error rate.

Failure Case 1 — False Negative: diluted toxicity (BERT confidence: 0.974)

```
Input:  [Long civic complaint about a building lease dispute,
concluding
        with: "...This article made me scream out at breakfast
this
        morning, 'Motherf**ers!'" ]
True Label: Toxic | Prediction: Non-Toxic | Confidence: 97.4%
Analysis: A 300-word non-toxic argument contains one expletive as an
        emotional aside. BERT's contextual window down-weights the
        single toxic signal relative to the overwhelming non-toxic
        context.
```

Failure Case 2 — False Positive: casual social media language (BERT confidence: 0.643)

```
Input:  "It seems to be no coincidence that these trolls appear
on weekends."
True Label: Non-Toxic | Prediction: Toxic | Confidence: 64.3%
Analysis: The word 'trolls' carries negative connotations that
surface
        in BERT's learned representations, despite the comment
being
        a legitimate, civil observation. Low confidence (64.3%)
correctly
        signals model uncertainty on this borderline case.
```

We see that these two qualitative failure types are consistent with our expectation that the errors made by ToxiGuard lie at the fuzzy boundary between context-dependent toxicity and innocent discourse -- that is, the boundary of hardest cases according to the problem definition. A review tier of humans could likely catch a large proportion of these cases for a modest cost by applying a

confidence gate below 70% of a confidence score, as both error examples result in the classifier with relatively low confidence scores (97.4% and 64.3% respectively).

5. Discussion

5.1 Interpretation of Results

Overall, the results of this experiment reveal an interpretable picture of toxicity in multi-source online text. While, at the surface level, toxicity can be associated with a few distinct features (most notably negative sentiment and short length), at the same time, these features limit the performance, making them insufficient without the use of more contextual representations.

Sentiment negativity has the strongest correlation with toxicity of any of the studied features (Pearson $r = 0.46$), and matches previous work of Thomas Davidson on the prevalence of negative affect in offensive language. The strength of this correlation, however, is equally as informative, as it suggests that not all toxic text is characterized by overt negativity. Sarcasm, irony, or indirectly hostile messages can possess either a neutral or a superficially positive tone, making them unidentifiable with surface-level features alone. This highlights the aforementioned gap between classical models and contextual ones and backs up the claims made by Jacob Devlin about bidirectional transformer models' abilities.

The strong linear performance of the linear SVM (94.29% accuracy, 0.977 AUC) implies that the engineered feature space is, to a reasonable extent, linearly separable. Even with its low dimensionality, the feature set manages to capture the dominant signals inherent in toxic text: features based on length and sentiment. Although Random Forest yielded better recall, it appears that non-linear feature interactions do not add much significant value and that the linear SVM's ability to retain high precision and overall accuracy outweighs it.

While the increase in performance with the fine-tuned BERT (96.09%) might seem modest numerically, it represents a step up in the model's capabilities. Error analysis revealed that the misclassifications that remain mostly fall under categories of actually ambiguous and contextually toxic or figurate text, which supports Daniel Borkan's conclusions that such text is difficult to classify even for state-of-the-art systems.

Adding a generative GPT-2 component to the system goes beyond accuracy and serves as a qualitative measure. The ability to map different types of toxicity to relevant remedies demonstrates a higher level of understanding that extends beyond mere detection, which was supported by Robert Dale and Varvara Logacheva's work in controlled text generation.

5.2 Limitations

There are several limitations to the presented research.

First is the lack of domain generalization. Even though the dataset included data from various sources, it primarily comprises English-language text from a North American/European context, so its accuracy in other languages or communities (such as gaming chats or messenger apps) is likely to be lower and depends on domain specificity of toxicity models.

Second, toxicity detection was carried out with a binary classification scheme. Toxicity exists on a wide spectrum ranging from minor impoliteness to violent incitement, so reducing it to two categories obscures some nuances and may contribute to some of the misclassification outcomes. Using more fine-grained annotation systems like severity scales or multi-label classifications might be more helpful.

Third, no quantitative assessment of the generative component was performed. While some examples are promising, a full evaluation would require a human study based on several criteria (fluency, semantic similarity to original text, degree of toxicity removal, etc.). Existing measures such as style transfer accuracy, semantic similarity, and fluency were not implemented, which made comparison to other systems impossible.

Fourth, the template-driven approach in the generative component meant that although output explanations were consistent and coherent, they could also be robotic and monotonous. Creating more nuanced explanations would likely require a more complex taxonomy of toxicity, which in turn would need more data and a more advanced generative model. The current system thus can be seen as more of a proof-of-concept.

5.3 Ethical Considerations

Deployment of automatic toxicity detection mechanisms comes with significant ethical concerns, especially concerning free speech and community participation.

One issue is demographic bias. As studies by Daniel Borkan have shown, toxicity classifiers often perform worse for identity-related text associated with marginalized communities due to the nature of training data. The dataset in this experiment was also noted for its problematic training set (as described by Thomas Davidson) and the ToxiGuard model may suffer from similar issues. Performing formal bias audits is necessary to minimize risks.

The generative part carries a further concern regarding normative influence. By proposing changes in the text, the system seems to endorse a specific, likely more polite, communicative style that could be interpreted as overbearing or imposing in communities where directness and emotive language are the norm. Balancing protection from hate speech and respecting diverse communication styles is a difficult task.

Finally, automation of moderation is a contentious issue as even the most accurate classifiers will inevitably make mistakes (this experiment's error rate was 3.91%). This can affect user experience and can have unintended consequences. Human oversight is needed, especially in cases of low confidence and potentially to address borderline content. Mechanisms for appealing moderator decisions are also crucial.

6. Conclusion and Future Work

This project implemented ToxiGuard, an end-to-end system for detecting and explaining toxic comments as well as generating harmless alternatives. The system went through a methodical four-stage development process: from a baseline TF-IDF Logistic Regression, to an ensemble of fine-tuned classical classifiers that utilized engineered features; a fine-tuned BERT encoder for

classification; to a fine-tuned GPT-2 decoder that generated natural-language explanations and remediation.

Several insights can be drawn from the results of this project. Firstly, initial exploration of the dataset suggests that toxic language can indeed be represented by a small number of surface features - most importantly negative sentiment ($r = 0.46$) - along with text length and readability; combining these features together allows for performance by a classical SVM classifier to be comparable to modern deep learning methods (accuracy up to 94.29%) and indicate a high degree of linearity in the feature space. Secondly, fine-tuning a pre-trained BERT model achieved 96.09% accuracy and a macro F1-score of 0.9608, with 199 FPs and 214 FNs from 10554 testing data samples, respectively (which gives a balanced error distribution) - although the total increase over the best classical model was only 1.80 percent points, it does constitute a 32% decrease in the total error count and specifically highlights gains in detecting context-heavy, indirect toxicity. Thirdly, the generative model (GPT-2) has shown the ability to complement toxicity detection with explanations and constructive rewrites. When fine-tuned on a template, it can coherently explain the system's prediction, and it consistently provides a constructive alternative phrase for an input sentence classified as toxic, taking the system one step further

towards transparent and helpful content moderation. Overall, ToxiGuard is an interpretable and efficient system for toxicity detection, that has demonstrated the value of integrating classification with natural-language generation and bridging a key gap towards interactive moderation. Its modular structure (a BERT encoder for classification, and a GPT-2 decoder for generation) is designed to be adaptable for real-world applications, and permits independent model optimization.

There are a number of important future research directions stemming from this project. Firstly, thorough evaluation of the generative component with human annotations is critical; this requires use of standard methods such as STA, SIM, and FL, to facilitate comparison to prior detoxification work. Secondly, research into detecting and rectifying demographic bias is essential. Investigating false positive and false negative rates at subgroup levels will allow us to create more equitable models. Thirdly, evolving from binary classification to a more nuanced, multi-label, or severity based scheme could enable more precise analysis of the many forms that toxic language can take, and subsequently inform finer-grained explanation and remediation. Fourthly, improving classification performance further could involve employing more advanced or task-specific transformer architectures, such as RoBERTa or HateBERT, especially for the more challenging edge cases. Lastly, extending this system to a multilingual version will be a critical aspect of realizing its potential; recent advances in multilingual detoxification provide a clear path forward.

7. References

- Borkan, D., Dixon, L., Sorensen, J., Thain, N., & Vasserman, L. (2019). Nuanced Metrics for Measuring Unintended Bias with Real Data for Text Classification. In *Companion Proceedings of The 2019 World Wide Web Conference (WWW '19 Companion)*, pp. 491–500. ACM. <https://doi.org/10.1145/3308560.3317593>
- Dale, D., Voronov, A., Dementieva, D., Logacheva, V., Kozlova, O., Semenov, N., & Panchenko, A. (2021). Text Detoxification using Large Pre-trained Neural Models. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP 2021)*, pp. 7979–7996. Association for Computational Linguistics. <https://doi.org/10.18653/v1/2021.emnlp-main.629>

Davidson, T., Warmley, D., Macy, M., & Weber, I. (2017). Automated Hate Speech Detection and the Problem of Offensive Language. In *Proceedings of the 11th International AAAI Conference on Web and Social Media (ICWSM 2017)*, pp. 512–515. AAAI Press. <https://doi.org/10.1609/icwsm.v11i1.14955>

Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT 2019)*, Volume 1 (Long and Short Papers), pp. 4171–4186. Association for Computational Linguistics. <https://doi.org/10.18653/v1/N19-1423>

Hanu, L., & Unitary Team. (2020). Detoxify. GitHub repository. Available at: <https://github.com/unitaryai/detoxify>

Logacheva, V., Dementieva, D., Ustyantsev, S., Moskovskiy, D., Dale, D., Krotova, I., Semenov, N., & Panchenko, A. (2022). ParaDetox: Detoxification with Parallel Data. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL 2022)*, Volume 1 (Long Papers), pp. 6804–6818. Association for Computational Linguistics. <https://doi.org/10.18653/v1/2022.acl-long.469>

Nobata, C., Tetreault, J., Thomas, A., Mehdad, Y., & Chang, Y. (2016). Abusive Language Detection in Online User Content. In *Proceedings of the 25th International Conference on World Wide Web (WWW 2016)*, pp. 145–153. ACM. <https://doi.org/10.1145/2872427.2883062>

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, E. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, pp. 2825–2830.

Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., & Sutskever, I. (2019). Language Models are Unsupervised Multitask Learners. *OpenAI Blog*, 1(8). Available at: <https://openai.com/research/language-unsupervised>

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is All You Need. In *Advances in Neural Information Processing Systems (NeurIPS 2017)*, Volume 30. Curran Associates.